



PUCP

TEL141 - INGENIERÍA DE REDES CLOUD Hoja de Resultados

Entrega N°:

FINAL

2025

Tema:

Plan de Pruebas

Contents

Introducción	3
1. Módulo 1: User interface	4
1.1 Autenticación de usuario	4
1.2 Visualización de template disponibles	4
1.3 Creación de un nuevo template.....	4
1.4 Eliminación de template	4
1.5 Edición de template	4
2. Módulo 2: Slice Manager	4
2.1 Validación de estructura del JSON.....	4
2.2 Selección de infraestructura	5
3. Módulo 3: Linux Driver.....	6
3.1 Creación de VMs en Linux Cluster	6
3.2 Conectividad entre VMs del mismo slice	6
3.3 Eliminación de Slice	6
3.4 Asignación correcta de VNC/TAP/VM	6
4. Módulo 4: Openstack Driver	6
4.1 Creación de VMs en Linux Cluster	6
4.2 Conectividad entre VMs del mismo Slice	6
4.3 Eliminación de Slice 4.4 Obtención de URL para acceso de VNC a instancias.....	6
5. Módulo 5: VM Placement	7
5.1 Selección del host valida dentro de una Availability Zone	7
5.2 Selección correcta del host según la infraestructura elegida	7
6. Módulo 6: Persistence & Logging	7
6.1 Persistencia del estado del Slice en la Base de Datos	7
6.2 Registro de Logs al Crear/Eliminar slice	7

Introducción

El presente documento describe el Plan de Pruebas desarrollado para validar el funcionamiento integral del sistema cloud implementado como parte del proyecto. Su propósito principal es asegurar que cada componente opere de manera correcta, coherente y confiable frente a los distintos escenarios contemplados por el diseño.

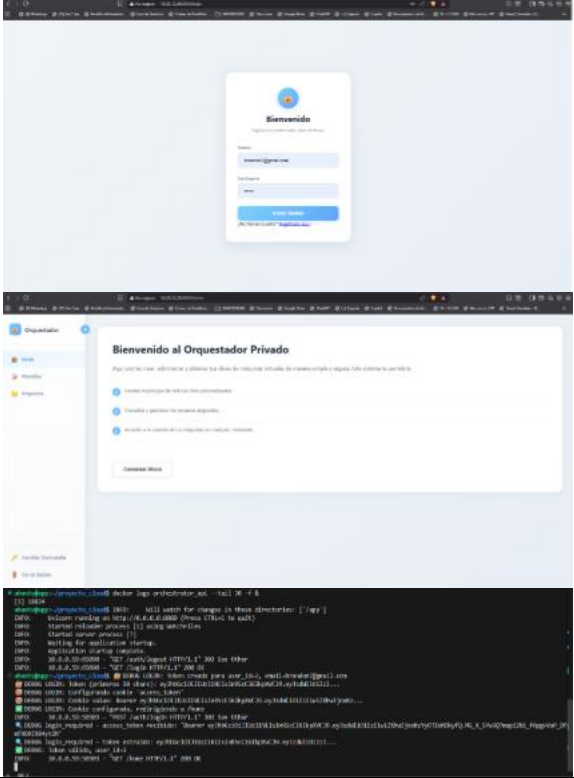
Las pruebas incluidas se enfocan en los módulos esenciales del orquestador: la validación del Slice Manager, la interacción con el clúster Linux, la integración con OpenStack, el funcionamiento del módulo de VM Placement y los mecanismos de persistencia y registro. Cada caso busca verificar que el sistema responda adecuadamente ante solicitudes válidas, garantizando no solo la creación y eliminación correcta de recursos, sino también la consistencia del estado interno y la trazabilidad de las acciones.

Finalmente, este plan establece una guía clara para evaluar la calidad global del sistema, proporcionando criterios, procedimientos reproducibles y resultados esperados para cada escenario. De esta manera, permite asegurar que la solución desarrollada cumple con los requerimientos funcionales, mantiene coherencia entre sus componentes y ofrece una base sólida para futuras ampliaciones o mejoras.

1. Módulo 1: User interface

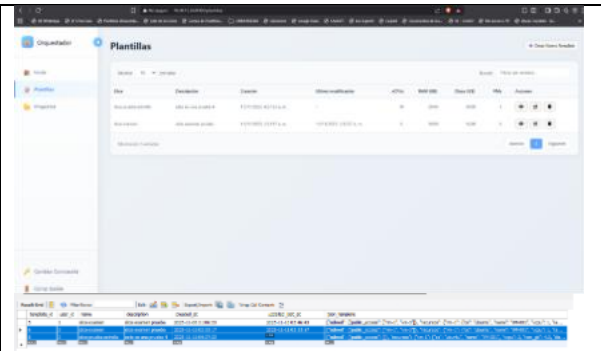
1.1 Autenticación de usuario

Número de caso	1.1																												
Objetivo del test	Verificar que el usuario pueda iniciar sesión correctamente desde la interfaz y que las credenciales inválidas sean rechazadas con un mensaje de error.																												
Diagrama de red del test	N/A																												
Prerrequisitos	-Entorno de pruebas desplegado en Docker. -BD operativa con usuarios de prueba creados. -Variables de entorno de autenticación configuradas.																												
Procedimiento	1. Ejecutar los contenedores con docker compose up -d --build. 2. Ingresar credenciales válidas. 3. Confirmar acceso al menú principal. 4. Repetir con credenciales incorrectas. 5. Observar mensaje de error.																												
Resultados esperados	- Con credenciales válidas, acceso exitoso y token JWT recibido. - Con credenciales inválidas, mensaje “Usuario o contraseña incorrectos”.																												
Resultado del test	PASS – El usuario pudo iniciar sesión con credenciales válidas, se generó el token JWT, se configuró la cookie access_token y se redirigió correctamente a /home. Con credenciales inválidas se mostró el mensaje “Usuario o contraseña incorrectos” FAIL – El comportamiento no coincide con lo esperado: no se genera el token JWT o no se configura la cookie, no se redirige a /home con credenciales válidas, o no aparece el mensaje “Usuario o contraseña incorrectos” al ingresar credenciales inválidas. NT – Si es que el ambiente no estaba disponible o había cambios pendientes.																												
Evidencias	 The screenshot shows the terminal output of a Docker Compose build command. The output includes the build process for the 'auth' service, which involves pulling the 'python:3.9-slim' image and installing dependencies. The build is successful. Below the terminal output, there is a table summarizing the test results. <table><thead><tr><th>id</th><th>user_id</th><th>email</th><th>password</th><th>token</th><th>access_token</th><th>refresh_token</th></tr></thead><tbody><tr><td>1</td><td>1</td><td>brandon@gmail.com</td><td>brandon</td><td>eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...</td><td>eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...</td><td>eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...</td></tr><tr><td>2</td><td>2</td><td>brandon@gmail.com</td><td>brandon</td><td>eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...</td><td>eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...</td><td>eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...</td></tr><tr><td>3</td><td>3</td><td>brandon@gmail.com</td><td>brandon</td><td>eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...</td><td>eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...</td><td>eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...</td></tr></tbody></table>	id	user_id	email	password	token	access_token	refresh_token	1	1	brandon@gmail.com	brandon	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...	2	2	brandon@gmail.com	brandon	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...	3	3	brandon@gmail.com	brandon	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...
id	user_id	email	password	token	access_token	refresh_token																							
1	1	brandon@gmail.com	brandon	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...																							
2	2	brandon@gmail.com	brandon	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...																							
3	3	brandon@gmail.com	brandon	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...																							

	
Comentarios y/o consideraciones	Validado con 3 usuarios de roles distintos.

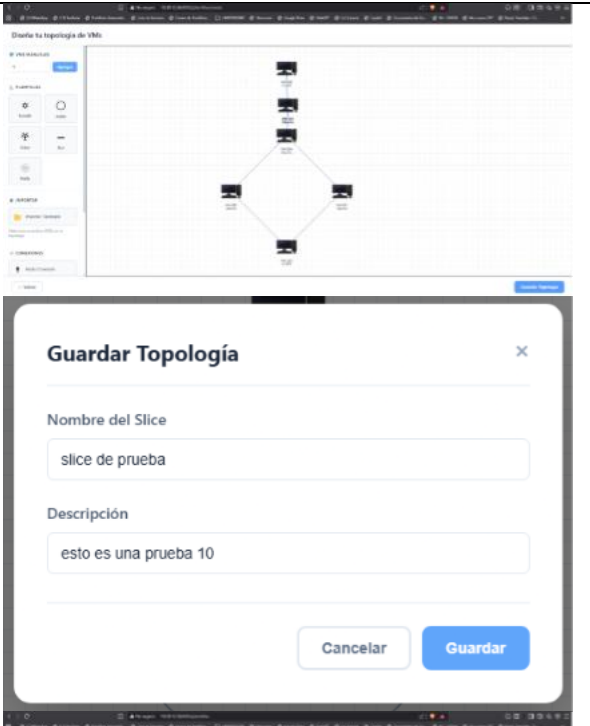
1.2 Visualización de template disponibles

Número de caso	1.2
Objetivo del test	Comprobar que la UI lista correctamente los templates almacenados en el backend y los muestra con sus atributos principales.
Diagrama de red del test	Cliente UI → API REST → Base de datos.
Prerrequisitos	- Usuario autenticado. - Base de datos con al menos 2 templates registrados.
Procedimiento	1. En la interfaz principal, seleccionar “Templates”. 2. Esperar la carga de la lista. 3. Se muestran los templates con atributos principales como el nombre, descripción, fechas de creación y modificación y por último los recursos asignados por topología
Resultados esperados	La UI muestra todos los templates existentes con información correcta y sin duplicados.
Resultado del test	PASS – La UI lista todos los templates almacenados en la base de datos, mostrando nombre, descripción, fechas y recursos asignados sin errores ni duplicados. FAIL – La UI no muestra todos los templates existentes o presenta información incorrecta/duplicada (faltan registros, campos vacíos o valores erróneos en los atributos). NT – Si es que no se disponía de usuario autenticado o no había templates en la BD.

Evidencias	
Comentarios y/o consideraciones	Se recomienda paginar si existen más de 10 templates.

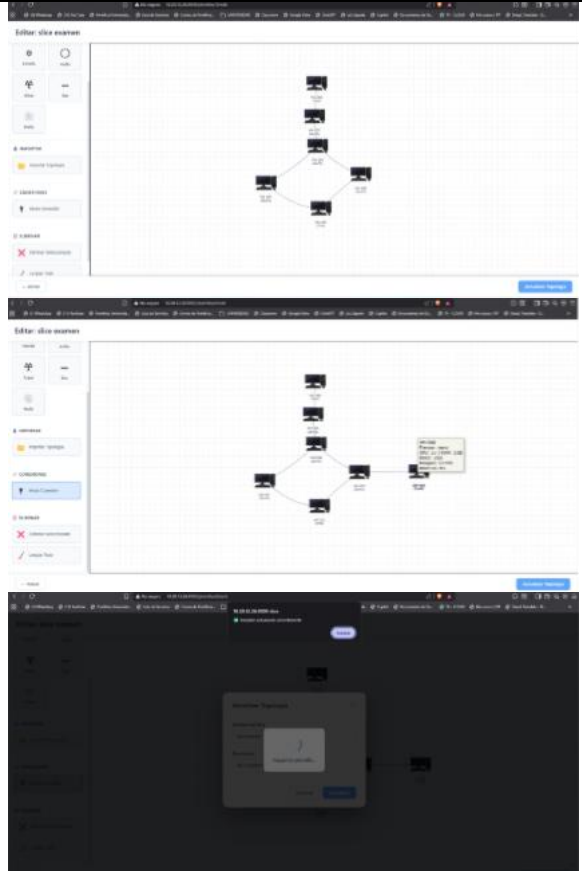
1.3 Creación de un nuevo template

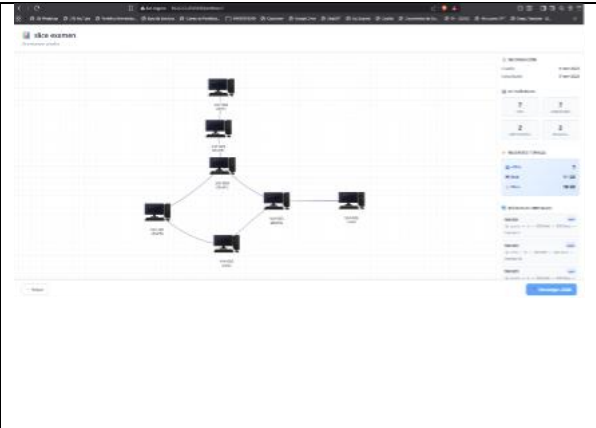
Número de caso	1.3
Objetivo del test	Validar que la creación de un nuevo template desde la UI registra los parámetros correctamente en el backend.
Diagrama de red del test	Cliente UI → Gateway → Base de datos.
Prerrequisitos	- Usuario autenticado. - Backend disponible.
Procedimiento	1. Seleccionar la sección “Plantillas” de la barra lateral. 2. Dentro puedes diseñar la plantilla que desees, con ayuda de topologías definidas, además que es posible realizar y borrar nuevas conexiones entre vms. 3. Se puede seleccionar los recursos respectivos de cada vm mediante flavours 4. Click a guardar topología
Resultados esperados	- Nuevo template registrado y visible. - Confirmación “Template creado exitosamente”.
Resultado del test	PASS – Se creó el nuevo template con los parámetros ingresados, se almacenó en el backend, aparece en la lista de templates y se muestra el mensaje “Template creado exitosamente”. FAIL – El template no se creó correctamente: no se almacenó en el backend, no aparece en la lista, no se mostró el mensaje de confirmación o se presentó un error durante el guardado. NT – Si es que no fue posible ser ejecutado (backend no disponible o el usuario no estaba autenticado).

Evidencias	 
Comentarios y/o consideraciones	- Nuevo template registrado y visible. - Confirmación “Template creado exitosamente”.

1.4 Eliminación de template

Número de caso	1.4
Objetivo del test	Comprobar que la UI permite eliminar un template y actualizar la lista sin errores.
Diagrama de red del test	Cliente UI → Gateway → Base de datos.
Prerrequisitos	- Usuario autenticado. - Template existente en la lista.
Procedimiento	1. Selección "Plantillas" 2. Clic en el icono de tachito para poder eliminar el template. 3. Confirmar acción. 4. Verificar actualización de lista.
Resultados esperados	El template desaparece de la lista.
Resultado del test	PASS – La UI permite eliminar el template seleccionado, se confirma la acción y la lista se

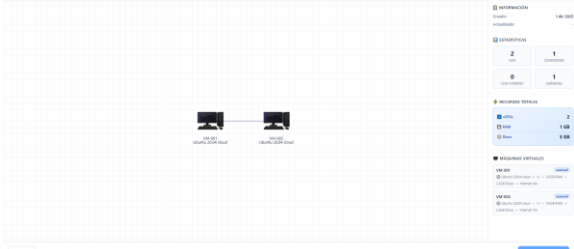
	número de VMs, SO, etc) y que los cambios se guarden.
Diagrama de red del test	Cliente UI → Gateway → Base de datos.
Prerrequisitos	- Template existente. - Usuario autenticado. - Gateway activo.
Procedimiento	1. Seleccionar un template existente. 2. Presionar “Editar”. 3. Cambiar parámetros (#VMs, SO). 4. Guardar y verificar lista actualizada.
Resultados esperados	- Cambios reflejados en la lista. - Mensaje “Template actualizado correctamente”.
Resultado del test	PASS – Los cambios realizados sobre el template (nombre, número de VMs, SO, etc.) se guardan correctamente, se muestran actualizados en la lista y aparece el mensaje “Template actualizado correctamente”. FAIL – Los cambios del template no se guardan o no se reflejan en la lista (no se actualizan los campos, se produce un error al guardar o no se muestra el mensaje de confirmación). NT – Si es que no se puede realizar ya que no se disponía de un template existente o el gateway no estaba activo
Evidencias	

	
Comentarios y/o consideraciones	Confirmar que el template ha sido actualizado correctamente.

2. Módulo 2: Slice Manager

2.1 Validación de estructura del JSON

Número de caso	2.1
Objetivo del test	Validar que el Slice Manager rechaza solicitudes con JSON incompleto, malformado o con campos no permitidos, y acepta JSON correctamente estructurado según el formato esperado por el orquestador.
Diagrama de red del test	UI → Slice Manager
Prerrequisitos	● Ingreso con usuario válido al UI ● Tener un template con JSON valido y otro con parámetros modificados o alterados
Procedimiento	1. Enviar la solicitud POST con el JSON válido 2. Verificara respuesta del Slice Manager 3. Enviar JSON no compatible 4. Registrar las respuestas de cada caso
Resultados esperados	● JSON válido → Código 201 / 200 con aceptación correcta. ● JSON invalido → Código 400 "JSON error".
Resultado del test	PASS: El Slice Manager recibe un JSON válido y responde con código 200/201, crea el registro correspondiente y continúa el flujo normal. Para los JSON inválidos, devuelve correctamente 400 Bad Request FAIL: El Slice Manager acepta un JSON inválido, crea un slice incompleto, no devuelve el error correspondiente, o se produce un estado

	inconsistente (slice parcialmente creado). También se considera FAIL si no diferencia entre un JSON válido e inválido o si devuelve un código HTTP incorrecto. NT: La prueba no se realizó por indisponibilidad del API, del Gateway, del Slice Manager o porque no se disponía de JSONs válidos/ inválidos preparados en el entorno de pruebas.
Evidencias	Descarga de JSON de un template  <pre>1 { 2 "subred": { 3 "public_access": [] 4 }, 5 "recursos": { 6 "vm-1": { 7 "os": "ubuntu20", 8 "name": "VM-001", 9 "vcpu": 1, 10 "ram_gb": 0.5, 11 "disk_gb": 2.5, 12 "flavour": "examenF" 13 }, 14 "vm-2": { 15 "os": "ubuntu20", 16 "name": "VM-002", 17 "vcpu": 1, 18 "ram_gb": 0.5, 19 "disk_gb": 2.5, 20 "flavour": "examenF" 21 } 22 }, 23 "topologia": { 24 "edges": [25 { 26 "id": "edge-vm-1-vm-2", 27 "to": "vm-2", 28 "from": "vm-1" 29 } 30], 31 "nodes": [32 { 33 "x": -70.63019807037772, 34 "y": 233.7238262330975, 35 "id": "vm-1", 36 "image": "/static/images/pc.png", 37 "label": "VM-001", 38 "shape": "image" 39 }, 40 { 41 "x": 149.36080192962228, 42 "y": 233.7238262330975, 43 "id": "vm-2", 44 "image": "/static/images/pc.png", 45 "label": "VM-002", 46 "shape": "image" 47 } 48] 49 } 50 }</pre> Pruebas con json válido e invalido
Comentarios y/o consideraciones	En este caso verificamos que el slice manager aplica las validaciones necesarias para entregar el formato a los drivers

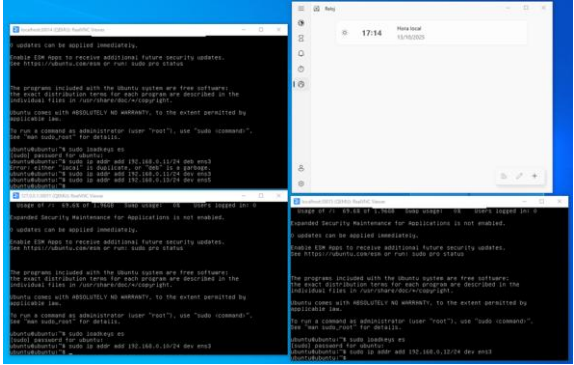
2.2 Selección de infraestructura

Número de caso	2.2
Objetivo del test	Verificar que el slice manager selecciona correctamente la infraestructura correspondiente (Linux Cluster u Openstack) según el host indicado por el módulo de VM Placement.
Diagrama de red del test	UI → Slice Manager → Módulo VM Placement → (Linux Driver / OpenStack Driver)
Prerrequisitos	• Módulo de VM Placement con el algoritmo en funcionamiento. • JSON que incluya la zona donde se va a colocar la VM
Procedimiento	1. Enviar solicitud de creación de slice con parámetro de zona de despliegue. 2. Verificar que el Slice manager envíe el JSON al driver correcto.
Resultados esperados	• Driver Linux recibe el JSON con la zona y la infraestructura adecuada • Driver Openstack recibe el JSON con la zona y la infraestructura adecuada
Resultado del test	PASS: El módulo VM Placement retorna un host, y el Slice Manager invoca correctamente el driver correspondiente: • host Linux → Linux Cluster Driver • host OpenStack → OpenStack Driver Los logs muestran la selección correcta y no se invoca el driver equivocado. FAIL: El Slice Manager invoca el driver incorrecto (por ejemplo, elige OpenStack cuando el host pertenece a Linux, o viceversa). También es FAIL si el Slice Manager ignora la salida del VM Placement o si no invoca ningún driver. NT: No se pudo ejecutar la prueba debido a que el módulo VM Placement no estuvo disponible, no había hosts registrados en la BD, o la API del Slice Manager no estaba operativa en el momento del test.
Evidencias	(Response JSON del placement, logs del Slice Manager mostrando qué driver se utilizó)
Comentarios y/o consideraciones	Este caso garantiza coherencia entre la decisión del VM Placement y la infraestructura realmente utilizada.

3. Módulo 3: Linux Driver

3.1 Creación de VMs en Linux Cluster

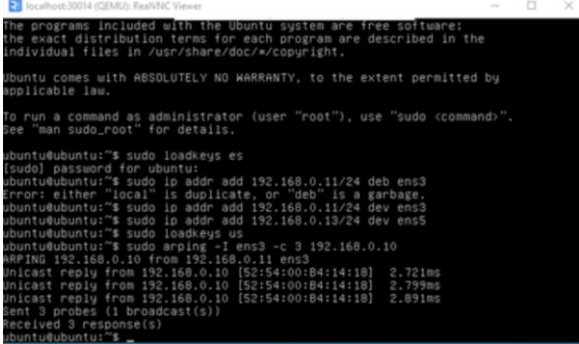
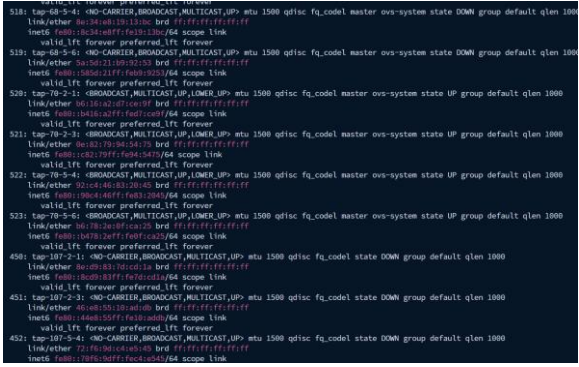
Número de caso	3.1
Objetivo del test	Validar que el Linux Cluster Driver crea correctamente las máquinas virtuales definidas en un slice, generando sus TAPs, asignando su puerto VNC, conectándolas a br-int y asegurando que cada VM arranca correctamente en el worker correspondiente.
Diagrama de red del test	UI / Slice Manager → Linux Driver → Workers(server1/server2/server3) → QEMU/KVM + OVS (br-int, TAP, VLAN)
Prerrequisitos	● Slice Manager en funcionamiento. ● Servidores accesibles por SSH ● Imágenes base disponibles en cada worker (ej.: ubuntu.img, cirros.img). ● Suficiente capacidad en CPU, RAM y disco en cada worker.
Procedimiento	1. Enviar desde la UI o Postman la solicitud de creación de un slice con backend Linux Cluster. 2. El Slice Manager envía el JSON al Linux Driver. 3. El Linux Driver selecciona el worker asignado y ejecuta la creación de la VM. 4. Validar en el worker que se creó un proceso QEMU por cada VM (ps aux grep qemu). 5. Validar que cada VM tiene su interfaz TAP creada (ip link show). 6. Validar que cada TAP está correctamente asociado a su bridge OVS (ovs-vsctl show). 7. Validar que los puertos VNC no están en conflicto.
Resultados esperados	● Todas las VMs definidas en el slice se crean en el worker asignado. ● Existe un proceso QEMU por VM. ● Cada VM tiene su interfaz TAP. ● No existe conflicto de puertos VNC. ● El Linux Driver retorna SUCCESS.
Resultado del test	PASS: VMs creadas, proceso QEMU activo, TAPs en OVS y VNC OK. FAIL: Falta VM, TAP no creado, OVS mal configurado o conflicto VNC. NT: Slice Manager o workers no disponibles.

Evidencias	● Capturas de ps aux grep qemu. <pre>root@ubuntu1:~# ps aux grep qemu root 38032 0.0 0.0 20472 20472 ? Ss 11:03 0:00 qemu-system-x86_64 -enable-kvm -s 2048 -m 1 -hda /img/ubuntu-22.04-amd64-vmdisk.tgz,if=tap-70-2-1,throttle=70-2-1,script= img.denywrite,qemu: nbd tap-70-2-1,throttle=70-2-1,script=,device=cdrom,servertap-70-2-1,acd=34,acd=1170,device=cdrom,servertap-70-2-1,acd=34,acd=1170 root 38033 1.4 1.4 20472 20472 ? Ss 11:03 0:00 qemu-system-x86_64 -enable-kvm -s 2048 -m 1 -hda /img/ubuntu-22.04-amd64-vmdisk.tgz,if=tap-70-2-1,throttle=70-2-1,script= img.denywrite,qemu: nbd tap-70-2-1,throttle=70-2-1,script=,device=cdrom,servertap-70-2-1,acd=34,acd=1170,device=cdrom,servertap-70-2-1,acd=34,acd=1170 root 38034 0.0 0.0 8208 20472 pts/1 Ss 11:03 0:00 grep --color=auto qemu root@ubuntu1:~#</pre> ● Capturas de TAPs creados (ip a). <pre>ubuntu@server2:~\$ ip a 52: tap-67-2-1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000 link/ether fa:16:00:58:00:00:00 brd ff:ff:ff:ff:ff:ff inet6 fe80::fa16:0058:0000:0000 scope link valid_lft forever preferred_lft forever 513: tap-67-2-1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000 link/ether fa:16:00:58:00:00:00 brd ff:ff:ff:ff:ff:ff inet6 fe80::fa16:0058:0000:0000 scope link valid_lft forever preferred_lft forever 1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000 link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00 inet 127.0.0.1/8 scope host lo valid_lft forever preferred_lft forever inet6 ::1/128 scope host valid_lft forever preferred_lft forever 514: vsw-system: <BROADCAST,MULTICAST,UP> mtu 1500 qdisc noop state DOWN group default qlen 1000 link/ether 86:14:7b:a1:32:19 brd ff:ff:ff:ff:ff:ff link/ether fa:16:00:58:00:00:00 brd ff:ff:ff:ff:ff:ff inet 192.168.201.1/24 brd 192.168.201.255 scope global vsw-system valid_lft forever preferred_lft forever inet6 fe80::f836:3aff:f82a:5a64 scope link valid_lft forever preferred_lft forever 515: br-int: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN group default qlen 1000 link/ether 86:14:7b:a1:32:19 brd ff:ff:ff:ff:ff:ff inet6 fe80::f836:3aff:f82a:5a64 scope link valid_lft forever preferred_lft forever 3: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel master vsw-system state UP group default qlen 1000 link/ether fa:16:00:58:00:00:00 brd ff:ff:ff:ff:ff:ff inet6 fe80::f836:3aff:f82a:5a64 scope link valid_lft forever preferred_lft forever 516: tap-68-2-1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel master vsw-system state DOWN group default qlen 1000 link/ether 86:14:7b:a1:32:19 brd ff:ff:ff:ff:ff:ff inet6 fe80::f836:3aff:f82a:5a64 scope link valid_lft forever preferred_lft forever</pre> ● Capturas del estado de ovs-vsctl show <pre>ubuntu@server1:~\$ sudo ovs-vsctl show 44a77839-8a54-43e0-8ad6-4ccb8b9827e5 Bridge br-int Port br-int Port tap-70-1-4 tag: 283 Interface tap-70-1-4 Port br-int Interface br-int type: internal Port tap-68-1-2 tag: 280 Interface tap-68-1-2 Port tap-68-1-w tag: 180 Interface tap-68-1-w Port tap-70-4-5 tag: 284 Interface tap-70-4-5 Port tap-70-4-1 tag: 283 Interface tap-70-4-1 Port tap-68-1-4 tag: 283 Interface tap-68-1-4 Port tap-68-4-1 tag: 283 Interface tap-68-4-1 Port tap-70-1-2 tag: 280 Interface tap-70-1-2 Port tap-70-1-w tag: 180 Interface tap-70-1-w</pre> ● Captura del acceso VNC. 
Comentarios y/o consideraciones	Validar que no existan procesos QEMU o TAPs residuales de pruebas anteriores.

3.2 Conectividad entre VMs del mismo slice

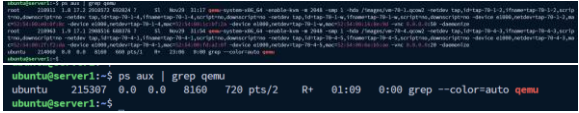
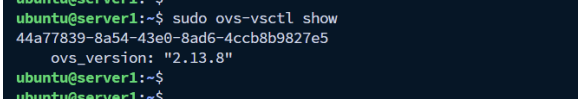
Número de caso	3.2
Objetivo del test	Validar que las VMs desplegadas dentro de un mismo slice, incluso si se encuentran distribuidas en distintos workers del clúster Linux, pueden comunicarse correctamente a

	nivel de capa 2 y capa 3 a través de los bridges OVS y las VLAN asignadas por el Slice Manager.
Diagrama de red del test	VM1 (Worker A) ↔ OVS br-int ↔ VLAN ↔ OVS br-ofs ↔ OVS br-int ↔ VM2 (Worker B)
Prerrequisitos	● Slice desplegado con al menos 2 VMs pertenecientes al mismo slice. ● Workers operativos y accesibles. ● Interfaces TAP conectadas al bridge correspondiente. ● OVS funcionando como switch troncal. ● VLAN del slice correctamente propagada en los workers. ● IPs internas asignadas vía DHCP o configuradas correctamente.
Procedimiento	1. Identificar las dos VMs pertenecientes al mismo slice. 2. Obtener sus direcciones IP internas asignadas. 3. Realizar ping a la maquina correspondiente. 4. Ver la captura del trafico con: sudo ovs-tcpdump -i br-int sudo tcpdump -i <ensX>
Resultados esperados	● Ambas VMs responden ping exitosamente en ambas direcciones. ● Se observan paquetes ARP e ICMP atravesando: - br-int de cada worker - br-ofs (switch central) - trunks VLAN del slice ● No se presentan pérdidas de paquetes ni timeouts.
Resultado del test	PASS: La comunicación entre las VMs del slice funciona correctamente. FAIL: Las VMs no tienen conectividad L2/L3 o no se observan paquetes en OVS. NT: No se realizó la prueba por indisponibilidad del Slice Manager, workers, OVS o parámetros inválidos del slice.
Evidencias	● Captura del airping exitoso entre las dos VMs.

	 <pre> The programs included with the Ubuntu system are free software; the exact distribution terms for each program are described in the individual files in /usr/share/doc/*/copyright. Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by applicable law. To run a command as administrator (user "root"), use "sudo <command>". See "man sudo_root" for details. ubuntu@ubuntu:~\$ sudo loadkeys es [sudo] password for ubuntu: ubuntu@ubuntu:~\$ sudo ip addr add 192.168.0.11/24 dev ens3 Error: either "local" is duplicate, or "deb" is a garbage. ubuntu@ubuntu:~\$ sudo ip addr add 192.168.0.11/24 dev ens3 ubuntu@ubuntu:~\$ sudo loadkeys us ubuntu@ubuntu:~\$ sudo arping -I ens3 -c 3 192.168.0.10 ARPING 192.168.0.10 from 192.168.0.11 ens3 Unicast reply from 192.168.0.10 [52:54:00:B4:14:18] 2.721ms Unicast reply from 192.168.0.10 [52:54:00:B4:14:18] 2.799ms Unicast reply from 192.168.0.10 [52:54:00:B4:14:18] 2.631ms Sent 3 probes (1 broadcast(s)) Received 3 response(s) ubuntu@ubuntu:~\$ _ </pre> • Capturas de tcpdump mostrando tráfico ARP/ICMP en - br-int (en ambos workers) - br-ofs (en OFS) • Captura del comando “ip a” mostrando TAPs asociados.  <pre> 518: tap-68-5-4: <BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel master ovs-system state DOWN group default qlen 1000 link/ether 8a:34:a0:10:13:0e brd ff:ff:ff:ff:ff:ff inet6 fe80::8a34:a0ff:fe10:130e/64 scope link valid_lft forever preferred_lft forever 519: tap-68-5-6: <BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel master ovs-system state DOWN group default qlen 1000 link/ether 5a:5d:21:00:52:53 brd ff:ff:ff:ff:ff:ff inet6 fe80::5a5d:21ff:fe00:5253/64 scope link valid_lft forever preferred_lft forever 520: tap-79-2-1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel master ovs-system state UP group default qlen 1000 link/ether 3a:15:02:00:00:00 brd ff:ff:ff:ff:ff:ff inet6 fe80::3a15:02ff:fe00:0000/64 scope link valid_lft forever preferred_lft forever 521: tap-79-2-3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel master ovs-system state UP group default qlen 1000 link/ether 6c:82:79:3d:54:79 brd ff:ff:ff:ff:ff:ff inet6 fe80::6c82:79ff:fe3d:5479/64 scope link valid_lft forever preferred_lft forever 522: tap-79-5-4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel master ovs-system state UP group default qlen 1000 link/ether 22:c4:86:03:20:0c brd ff:ff:ff:ff:ff:ff inet6 fe80::22c4:86ff:fe03:200c/64 scope link valid_lft forever preferred_lft forever 523: tap-79-5-6: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel master ovs-system state UP group default qlen 1000 link/ether 3d:7b:2e:bf:ca:20 brd ff:ff:ff:ff:ff:ff inet6 fe80::3d7b:2eff:febf:ca20/64 scope link valid_lft forever preferred_lft forever 450: tap-187-2-1: <BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000 link/ether 8c:0b:21:10:c0:1a brd ff:ff:ff:ff:ff:ff inet6 fe80::8c0b:21ff:fe10:c01a/64 scope link valid_lft forever preferred_lft forever 451: tap-187-2-3: <BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000 link/ether 45:a8:55:38:a0:00 brd ff:ff:ff:ff:ff:ff inet6 fe80::45a8:55ff:fe38:a000/64 scope link valid_lft forever preferred_lft forever 452: tap-187-5-4: <BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000 link/ether 72:10:00:00:00:00 brd ff:ff:ff:ff:ff:ff inet6 fe80::7210:00ff:fe00:0000/64 scope link </pre>
Comentarios y/o consideraciones	• Verificar conectividad tanto en VMs ubicadas en el mismo worker como en workers distintos. • Validar que la VLAN del slice esté siendo propagada correctamente. • Verificar que no existan TAPs residuales de pruebas anteriores que interfieran con OVS.

3.3 Eliminación de Slice

Número de caso	3.3
Objetivo del test	Verificar que el Linux Driver elimina correctamente todas las VMs pertenecientes al slice, detiene los procesos QEMU/KVM, elimina las interfaces TAP asociadas, remueve los puertos del bridge OVS y libera los puertos VNC utilizados por dichas VMs.
Diagrama de red del test	Slice Manager → Linux Driver → Workers (server2/server3/server4) → QEMU/KVM + OVS (br-int, TAP)

Prerrequisitos	● Slice previamente desplegado en el backend Linux. ● Workers operativos y accesibles mediante SSH. ● Slice Manager ejecutándose de manera estable. ● No deben existir fallos previos en la creación del slice.
Procedimiento	1. Ejecutar desde la UI la solicitud de eliminar slice. 2. El Slice Manager envía la orden de borrado al Linux Driver. 3. En los workers implicados, verificar: Que los procesos QEMU correspondientes fueron detenidos(ps aux grep qemu) 4. Validar que los puertos VNC asociados ya no están escuchando.
Resultados esperados	● No queda ningún proceso QEMU activo del slice. ● Todos los TAPs del slice han sido eliminados. ● No quedan puertos VNC ocupados por ese slice.
Resultado del test	PASS: Todos los recursos del slice (QEMU, TAPs, OVS, VNC) fueron eliminados correctamente en cada worker. FAIL: Persisten procesos QEMU, TAPs, puertos OVS o puertos VNC tras la operación de borrado. NT: No se ejecutó la prueba por no existir slices activos, por indisponibilidad del Linux Driver o por fallos previos en el despliegue.
Evidencias	● Capturas del estado de QEMU antes y después.  <pre> ubuntu@server1:~\$ ps aux grep qemu ubuntu 215387 0.0 0.0 8160 728 pts/2 R+ 01:09 0:00 grep --color=auto qemu ubuntu@server1:~\$ </pre> ● Salida de ovs-vsctl show.  <pre> ubuntu@server1:~\$ sudo ovs-vsctl show 44a77839-8a54-43e0-8ad6-4ccb8b9827e5 ovs_version: "2.13.8" ubuntu@server1:~\$ </pre> ● Estado de TAPs (ip a).

	<pre> ubuntu@server1:~\$ ip a 1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000 link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00 inet 127.0.0.1/8 scope host lo valid_lft forever preferred_lft forever inet6 ::1/128 scope host valid_lft forever preferred_lft forever 2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000 link/ether fa:16:3e:4c:3c:37 brd ff:ff:ff:ff:ff:ff inet 192.168.101.1/24 brd 192.168.101.255 scope global ens3 valid_lft forever preferred_lft forever inet6 fe80::fa16:3eff:fe4c:3c37/64 scope link valid_lft forever preferred_lft forever 3: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel master vns-system state UP group default qlen 1000 link/ether fa:16:3e:4c:3c:37 brd ff:ff:ff:ff:ff:ff 685: tap-107-1-2: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000 link/ether 5a:0b:9c:7f:6a:10 brd ff:ff:ff:ff:ff:ff inet6 fe80::5a0b:9cff:fe7f:6a10/64 scope link valid_lft forever preferred_lft forever 686: tap-107-1-4: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000 link/ether c2:50:8e:a2:f2:a6 brd ff:ff:ff:ff:ff:ff inet6 fe80::c250:8eaf:f2a2:f2a6/64 scope link valid_lft forever preferred_lft forever 687: tap-107-1-w: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000 link/ether 8a:16:3e:4c:3c:37 brd ff:ff:ff:ff:ff:ff inet6 fe80::8a16:3eff:fe4c:3c37/64 scope link valid_lft forever preferred_lft forever 688: tap-107-4-3: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000 link/ether 66:65:0f:02:0f:fa brd ff:ff:ff:ff:ff:ff inet6 fe80::6665:0fff:020f:fa/64 scope link valid_lft forever preferred_lft forever </pre> ● Confirmación del estado del slice en BD. 
Comentarios y/o consideraciones	● Realizar un doble chequeo para asegurar que no queden procesos QEMU residuales ni TAPs huérfanos. ● Verificar workers múltiples si el slice fue distribuido. ● Si quedan TAPs o QEMU activos, se debe ejecutar limpieza manual y reportar falla.

3.4 Asignación correcta de VNC/TAP/VM

4. Módulo 4: Openstack Driver

4.1 Creación de VMs

Número de caso	4.1
Objetivo del test	Validar que el OpenStack Driver crea correctamente las VMs definidas en un slice, generando las instancias, puertos Neutron y redes necesarias dentro del proyecto asignado, y que el estado final de todas las instancias sea ACTIVE.
Diagrama de red del test	Slice Manager → OpenStack Driver → OpenStack APIs (Nova, Neutron, Glance) → Horizon / CLI
Prerrequisitos	● OpenStack funcional (Keystone, Nova, Neutron) en Headnode. ● Imagen y flavor previamente creados en OpenStack. ● Proyecto (tenant) creado para el slice.

	● JSON válido del slice a desplegar.
Procedimiento	1. Ejecutar la creación del slice desde UI. 2. El Slice Manager envía la orden al OpenStack Driver. 3. En el Headnode, verificar creación de instancias con: <code>openstack server list --project <project_name></code> 4. Verificar creación de redes/puertos asociados: <code>openstack network list --project <project_name></code> 5. Comprobar en Horizon que las instancias aparecen en Compute > Instances como ACTIVE. 6. Validar que la topología de red muestra correctamente las conexiones creadas.
Resultados esperados	● Todas las instancias definidas en el slice aparecen en server list. ● Los puertos y redes del slice son creados en el proyecto correspondiente. ● El estado final de todas las VMs es ACTIVE. ● Las configuraciones (imagen, flavor, nombre, red) coinciden con el JSON enviado. ● La topología gráfica en Horizon refleja correctamente el slice.
Resultado del test	PASS: Las instancias del slice se crean correctamente en OpenStack y todas alcanzan estado ACTIVE. FAIL: Alguna VM no aparece, queda en estado ERROR/BUILD infinito, faltan puertos o redes, o el Driver no crea los recursos según el JSON. NT: No se ejecutó la prueba por errores previos (proyecto no creado, imagen o flavor inexistente, Keystone caído).
Evidencias	● Salida de <code>openstack server list</code> antes/después. ● Salida de <code>openstack network list</code>, <code>openstack port list</code>. ● Capturas de Horizon mostrando las VMs en estado ACTIVE.
Comentarios y/o consideraciones	Verificamos que el proyecto usado corresponde al slice actual, y que no quedan recursos huérfanos si la creación falla parcialmente.

4.2 Conectividad entre VMs del mismo Slice

Número de caso	4.2
Objetivo del test	Validar que las VMs desplegadas en OpenStack dentro del mismo slice pueden comunicarse correctamente entre sí a nivel de capa 2 y capa 3.
Diagrama de red del test	VM1 (net-slice) ↔ Neutron Port ↔ OVS/OVN Internal Switch ↔ Neutron Port ↔ VM2 (net-slice)
Prerrequisitos	● Slice desplegado en OpenStack. ● Al menos dos VMs conectadas a la misma red Neutron. ● Instancias en estado ACTIVE. ● Acceso a Horizon o CLI de OpenStack
Procedimiento	1. Obtener las direcciones IP internas de ambas VMs: <code>openstack server list --project <project></code> 2. Acceder a una instancia vía consola VNC: <code>openstack console url show <vm_id></code> 3. Desde VM1 ejecutar: <code>ping <IP_VM2></code> 4. Desde VM2 ejecutar: <code>ping <IP_VM1></code> 5. Registrar tráfico ARP e ICMP entre ambas VMs.
Resultados esperados	● Ambas VMs responden ping entre sí sin pérdida de paquetes. ● Neutron otorga correctamente IPs internas. ● El tráfico ARP e ICMP es visible en la red interna. ● La topología en Horizon muestra ambas VMs unidas a la misma red.
Resultado del test	PASS: Las VMs del slice se comunican correctamente entre sí (ARP + ICMP) dentro de la red Neutron. FAIL: No hay respuesta de ping entre VMs. NT: No se completó el despliegue de las VMs.
Evidencias	● Capturas de <code>openstack server list</code>. ● Capturas de ping desde ambas VMs. ● Capturas de <code>tcpdump</code>. ● Capturas de la topología de Neutron en Horizon
Comentarios y/o consideraciones	También se valida que los puertos asociados a las vms estén en estado ACTIVE.

4.3 Eliminación de Slice

Número de caso	4.3
----------------	-----

Objetivo del test	Verificar que el OpenStack Driver elimina correctamente todas las instancias, puertos y recursos Neutron asociados al slice.
Diagrama de red del test	Slice Manager → OpenStack Driver → Nova → Neutron
Prerrequisitos	• Slice previamente desplegado OpenStack. • Instancias en estado ACTIVE. • Red y puertos del slice existentes.
Procedimiento	1. Desde la UI enviar solicitud de eliminar slice. 2. El Slice Manager envía orden de borrado al OpenStack Driver. 3. Validar eliminación de instancias: <code>openstack server list --project <project></code> 4. Validar eliminación de puertos: <code>openstack port list --project <project></code> 5. Confirmar que no existan redes o recursos residuales relacionados con el slice. 6. Corroborar en Horizon (Compute > Instances / Network > Topology) que no hay recursos visibles del slice.
Resultados esperados	• Todas las instancias del slice desaparecen de server list. • Todos los puertos Neutron asociados son eliminados. • No quedan recursos huérfanos (puertos, instancias, redes temporales). • El proyecto vuelve a su estado previo al despliegue.
Resultado del test	PASS: Todos los recursos del slice (instancias y puertos Neutron) fueron eliminados correctamente sin dejar huérfanos. FAIL: Persisten instancias en estado ERROR o SHUTOFF o la existencia de recursos huérfanos. NT: El slice no estaba desplegado.
Evidencias	• Capturas de <code>openstack server list</code> antes/después. • Capturas de <code>openstack port list</code>. • Capturas de topología en Horizon.
Comentarios y/o consideraciones	Igualmente tomar en cuenta con logs de salida al momento de ejecutarse el borrado.

4.4 Obtención de URL para acceso de VNC a instancias

Número de caso	4.4
Objetivo del test	Validar que el OpenStack Driver obtiene correctamente la URL de acceso VNC para cada instancia creada en el slice y que esta URL

	permite visualizar la consola de la VM mediante el cliente NoVNC o la interfaz web.
Diagrama de red del test	Slice Manager → OpenStack Driver → Nova API (os-getVNCConsole) → Cliente VNC / Horizon
Prerrequisitos	● Slice desplegado en backend OpenStack. ● Todas las instancias en estado ACTIVE. ● Nova Console API funcionando correctamente.
Procedimiento	1. Identificar los IDs de las VMs generadas por el slice: <code>openstack server list --project <project></code> 2. Para cada instancia, solicitar la URL de consola VNC: <code>openstack console url show <vm_id></code> 3. Verificar que la salida incluye un campos solicitados
Resultados esperados	● El comando <code>openstack console url show</code> retorna una URL válida por instancia. ● La consola NoVNC se abre sin errores y muestra la VM activa. ● El Slice Manager registra el URL o lo retorna a la API según diseño.
Resultado del test	PASS: Las URLs VNC fueron obtenidas correctamente y permiten acceder a la consola de todas las instancias del slice. FAIL: Alguna VM no retorna URL. NT: Las instancias no estaban en estado ACTIVE.
Evidencias	● Captura del comando <code>openstack console url show <vm_id></code>. ● Captura de la consola NoVNC con la VM activa. ● Logs del driver mostrando la obtención del URL.
Comentarios y/o consideraciones	Se sabe que Openstack también puede trabajar con <code>xvnc</code> , pero este caso solo se muestra el uso de <code>novnc</code> .

5. Módulo 5: VM Placement

5.1 Selección del host valida dentro de una Availability Zone

Número de caso	5.1
Objetivo del test	Validar que el módulo de VM Placement selecciona únicamente hosts pertenecientes a la Availability Zone solicitada en el template del slice, descartando cualquier host fuera de la zona incluso si posee recursos disponibles.

Diagrama de red del test	UI → Slice Manager → VM Placement → Driver correspondiente (Linux u OpenStack)
Prerrequisitos	• Hosts registrados en la base de datos. • Al menos un host válido dentro de la zona solicitada. • Slice Manager y módulo VM Placement operativos. • Template de slice que incluya el campo "zone": "<nombre_zona>".
Procedimiento	1. Consultar la lista de hosts registrados y verificar sus zonas 2. Enviar un JSON de creación de slice especificando 3. Observar la respuesta del módulo VM Placement: 4. Validar que el Slice Manager reciba un host cuya zona coincide con la solicitada.
Resultados esperados	• El módulo VM Placement filtra correctamente todos los hosts fuera de la zona. • El host seleccionado pertenece a la Availability Zone solicitada. • No se selecciona ningún host de otra zona, aunque tenga recursos. • El Slice Manager recibe un resultado coherente con el criterio de zona.
Resultado del test	PASS: El módulo seleccionó un host perteneciente a la Availability Zone solicitada y descartó correctamente los hosts de otras zonas. FAIL: El módulo seleccionó un host de una zona incorrecta. NT: No existían hosts registrados en la zona solicitada.
Evidencias	<pre> ubuntu@app:~/vm_placement\$ python3 test_casos_1_v2.py ===== 1) LISTA DE HOSTS REGISTRADOS Y SUS ZONAS (/nodes/status) ===== compute-node-server1 zone=AZ1 platform=linux compute-node-server2 zone=AZ2 platform=linux compute-node-server3 zone=AZ3 platform=linux compute-node-worker1 zone=AZ4 platform=openstack compute-node-worker2 zone=AZ5 platform=openstack compute-node-worker3 zone=AZ5 platform=openstack ===== 2) ANALISIS TEORICO SIN RESTRICION DE ZONA ===== Usando mismo slice pedido a la API: CPU reservada: 1 cores RAM reservada: 1.0 GB Perfil: Estudiante Contexto: Virtualizacion general Riesgo teorico por host (ignorando zona): ===== Host Zone Plat Pcpu Pram Pmax ----- compute-node-server1 AZ1 linux ~0 0.002730 0.002730 compute-node-server2 AZ2 linux ~0 2.81e-14 2.81e-14 compute-node-server3 AZ3 linux ~0 2.81e-13 2.81e-13 compute-node-worker1 AZ4 openstack 1.70e-189 8.30e-21 8.60e-21 compute-node-worker2 AZ5 openstack ~0 8.61e-186 8.61e-186 compute-node-worker3 AZ5 openstack 9.18e-314 1.59e-48 1.59e-48 ===== Host con MENOR riesgo teorico (ignorando zona y plataforma): -> compute-node-worker2 en zona AZ5 (platform=openstack), Pmax ~ 8.61e-186 </pre>

	<pre>===== 3) SOLICITUD REAL DE PLACEMENT PARA LA ZONA AZ4 ===== JSON enviado a /api/v1/placement: { "cpu": 1, "ram_gb": 1.0, "disk_gb": 1.0, "zone": "AZ4", "platform": "openstack", "user_profile": "Estudiante", "technical_context": "Virtualizacion general" } Codigo HTTP: 200 JSON de respuesta: { "placement": { "availability zone": "AZ4", "host": "compute-node-worker1", "platform": "openstack", "reason": "Host OpenStack en AZ AZ4 seleccionado con riesgo máximo 0.0000" }, "success": true } Comprobando coherencia de zona... [PASS] Host seleccionado: compute-node-worker1 (zone=AZ4) coincide con la zona solicitada (AZ4). Conclusion: - La API respeta la zona de disponibilidad solicitada (AZ4). ubuntu@app:~/vm_placement\$ █</pre>
Comentarios y/o consideraciones	Este caso verifica la coherencia lógica del sistema: la zona actúa como filtro primario antes de aplicar cualquier cálculo de recursos.

5.2 Selección correcta del host según la infraestructura elegida

Número de caso	5.2
----------------	-----

Objetivo del test	Verificar que el módulo de VM Placement selecciona correctamente un host cuya infraestructura (Linux Cluster u OpenStack) coincide con la infraestructura especificada o requerida en el slice, y que el Slice Manager invoca el driver adecuado según dicho host.
Diagrama de red del test	UI / API → Slice Manager → VM Placement → Linux Driver / OpenStack Driver
Prerrequisitos	● Slice Manager operativo. ● Template de slice válido (backend explícito o automático). ● VM Placement funcionando.
Procedimiento	1. Revisar la lista de hosts registrados y verificar su backend (Linux u OpenStack). 2. Enviar un JSON de creación de slice que requiera backend Linux o cuyos parámetros lleven a elegir un host Linux. 3. Verificar que el módulo VM Placement retorna un host Linux. 4. Revisar que el Slice Manager invoque Linux Driver. 5. Repetir la prueba enviando un slice cuyo placement lleve a elegir un host OpenStack.
Resultados esperados	● Cuando el placement retorna un host Linux → el Slice Manager debe usar Linux Driver. ● Cuando el placement retorna un host OpenStack → debe usarse OpenStack Drive. ● La respuesta JSON refleja correctamente el backend seleccionado.
Resultado del test	PASS: El host seleccionado pertenece al backend correcto y el Slice Manager invocó el driver correspondiente. FAIL: El placement selecciona un host del backend incorrecto. NT: Módulo VM Placement no operativo.

Evidencias

```
ubuntu@app:~/vm_placement$ python3 test_caso5_2_vm_placement.py
=====
1) LISTA DE HOSTS REGISTRADOS (/nodes/status)
=====
compute-node-server1 | zone=AZ1 | platform=linux
compute-node-server2 | zone=AZ2 | platform=linux
compute-node-server3 | zone=AZ3 | platform=linux
compute-node-worker1 | zone=AZ4 | platform=openstack
compute-node-worker2 | zone=AZ5 | platform=openstack
compute-node-worker3 | zone=AZ5 | platform=openstack
=====

2) SOLICITUD DE PLACEMENT PARA BACKEND LINUX EN ZONA AZ2
=====
JSON enviado a /api/v1/placement:
{
  "cpu": 1,
  "ram_gb": 1.0,
  "disk_gb": 1.0,
  "zone": "AZ2",
  "platform": "linux",
  "user_profile": "Estudiante",
  "technical_context": "Virtualizacion general"
}

Codigo HTTP: 200

JSON de respuesta:
{
  "placement": {
    "availability zone": "AZ2",
    "host": "compute-node-server2",
    "platform": "linux",
    "reason": "Host Linux seleccionado con riesgo máximo 0.0000"
  },
  "success": true
}

Comprobando coherencia de backend...
[PASS] La API devolvio host compute-node-server2 con platform=linux,
y en nodes_status.json ese host tambien es linux.

Comprobando coherencia de zona...
[PASS] Host compute-node-server2 pertenece a la zona solicitada (AZ2).
```

```
=====
2) SOLICITUD DE PLACEMENT PARA BACKEND OPENSTACK EN ZONA AZ4
=====
JSON enviado a /api/v1/placement:
{
  "cpu": 1,
  "ram_gb": 1.0,
  "disk_gb": 1.0,
  "zone": "AZ4",
  "platform": "openstack",
  "user_profile": "Estudiante",
  "technical_context": "Virtualizacion general"
}

Codigo HTTP: 200

JSON de respuesta:
{
  "placement": {
    "availability zone": "AZ4",
    "host": "compute-node-worker1",
    "platform": "openstack",
    "reason": "Host OpenStack en AZ AZ4 seleccionado con riesgo máximo 0.0000"
  },
  "success": true
}

Comprobando coherencia de backend...
[PASS] La API devolvio host compute-node-worker1 con platform=openstack,
y en nodes_status.json ese host tambien es openstack.

Comprobando coherencia de zona...
[PASS] Host compute-node-worker1 pertenece a la zona solicitada (AZ4).
ubuntu@app:~/vm_placement$
```

```
=====
NUEVA SOLICITUD DE PLACEMENT
=====
n: solicitud recibida: {cpu: 1, ram_gb: 1.0, disk_gb: 1.0, zone: 'AZ4', platform: 'openstack', user_profile: 'Estudiante', technical_context: 'Virtualizacion general'}

✓ Solicitud creada:
- CPU: 1 cores
- Mem: 1.0 GB
- Disco: 1.0 GB
- Zona: AZ4
- Plataforma: openstack
- Perfil: Estudiante
- Contexto: Virtualizacion general

% Consultando workers en zona: AZ4
✓ encontrados 1 workers en zona: AZ4
✓ compute-node-worker1 (openstack) - CPU: 4.0, RAM: 7.8GB

✓ Total de hosts válidos: 1
● Ejecutando algoritmo de placement...

=====
RIESGO ACTUAL DE HOSTS (ANTES DE ASIGNAR SLICE)
=====
compute-node-worker1 | Riesgo mínimo: 0.000000 | Zona: AZ4 | Plataforma: openstack
=====

EVALUANDO CANDIDATOS
=====
% Evaluando: compute-node-worker1
✓ Disco OK: 0.8 + 1.0 = 1.8 GB / 25.0 GB
● Riesgo después de asignar slice:
P(compartir CPU): 1.37878e2805158334e-151
P(compartir RAM): 1.9870871500050002e-17
■ CANDIDATO VIABLE - Riesgo máximo: 0.000000

=====
APLICANDO CRITERIO MINIMAX
=====
compute-node-worker1:
Riesgo local después: 0.000000
Riesgo global cluster: 0.000000
- Nuevo mejor candidato
```

	<pre> ===== COMPARATIVA: ANTES vs DESPUÉS DE LA ASIGNACIÓN ===== Host seleccionado: compute-node-worker1 ANTES de asignar el slice: Riesgo máximo del host: 0.000000 DESPUÉS de asignar el slice: P(congestión CPU): 0.000000 P(congestión RAM): 0.000000 Riesgo máximo del host: 0.000000 Riesgo global del clúster: 0.000000 ===== [+] PLACEMENT EXITOSO Host seleccionado: compute-node-worker1 Plataforma: openstack Zona: AZ4 Razon: Host OpenStack en AZ AZ4 seleccionado con riesgo máximo 0.0000 127.0.0.1 - - [01/Dec/2025 01:06:33] "POST /api/v1/placement HTTP/1.1" 200 - </pre>
Comentarios y/o consideraciones	Este caso asegura coherencia entre capa lógica (Placement) y capa operativa (Drivers), garantizando que el slice se despliega en la infraestructura correcta.

5.3 Rechazo de asignación por insuficiencia de recursos

Número de Caso	5.3
Objetivo del test	Validar que el módulo de VM Placement rechaza correctamente la creación de un slice cuando ningún host disponible posee los recursos suficientes
Diagrama de red del test	N/A
Prerrequisitos	-Hosts registrados en la base de datos (/nodes/status). -Slice Manager operativo. -Módulo VM Placement operativo. -Template de slice con recursos intencionalmente mayores a lo que cualquier host puede proveer (por ejemplo 64 cores y 64 GB RAM).
Procedimiento	1.Consultar /nodes/status antes del test Registrar: -CPU capacity / CPU mean -RAM capacity / RAM mean -Uso del host (current_usage) -Guardar los valores como estado_previo. 2.Enviar JSON de creación de slice con recursos imposibles, por ejemplo: <pre> { "cpu": 32, "ram_gb": 64, "disk_gb": 40, "zone": "AZ2", "platform": "linux", "user_profile": "Estudiante", "technical_context": "Virtualizacion general" } </pre> 3.Observar respuesta del VM Placement y debe devolver "success": false.

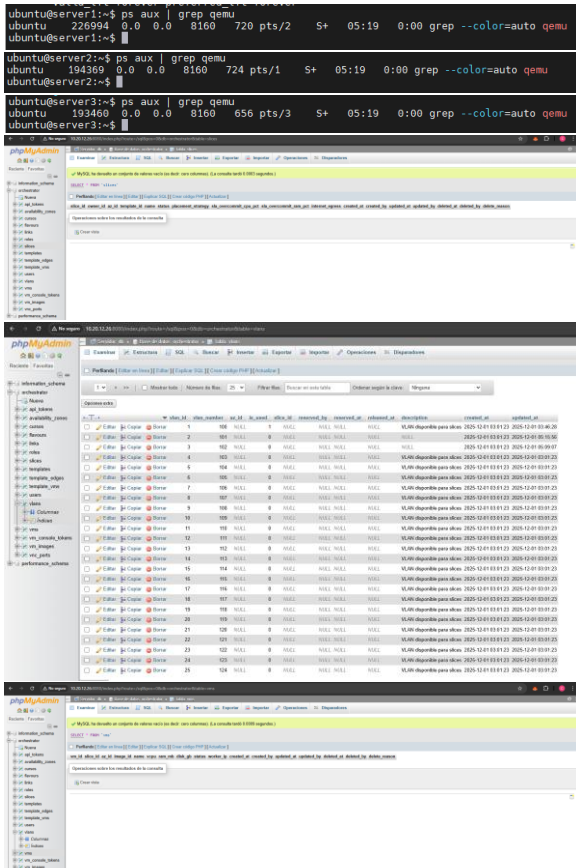
	4.Consultar nuevamente /nodes/status y guardar los valores como estado_posterior. 5.Comparar estado_previo vs estado_posterior 6.Validar que no se invocó ningún driver
Resultados esperados	-"success": false en la API. -Campo "error" indicando que no hay hosts con capacidad. -Ningún host altera sus métricas. -No se incrementa el número de VMs. -Drivers NO se ejecutan. -El estado del cluster antes y después del test es idéntico.
Resultado del Test	-PASS si la API rechaza la solicitud, devuelve error coherente y las métricas del cluster se mantienen idénticas antes y después del intento. -FAIL si la API asigna un host o cambia métricas aunque no debería. NT si no existen hosts registrados.
Evidencias	<pre> ubuntu@app:~/vm_placement\$ python3 generar_evidencias_nodos_status.py ===== GET /nodes/status (antes) ===== { "server1": { "cpu_capacity": { "unit": "cores", "value": 4 }, "current_usage": { "cpu": { "mean": 1.835645139376124, "std": 4.166504484892472, "unit": "%s" }, "disk": { "unit": "GB", "used": 7.1510712320000005 }, "ram": { "mean": 1.973854230797809, "std": 0.5105357954251593, "unit": "GiB" } }, "disk_capacity": { "unit": "GB", "value": 9.6 }, "enabled": true, "id": "server1", "in_maintenance": false, "metadata": { "datacenter": "Lima", "rack": "RACK-AZ1" }, "name": "compute-node-server1", "platform": "linux", "ram_capacity": { "unit": "GiB", "value": 3.8 }, "zone": "AZ1" }, "server2": { "cpu_capacity": { "unit": "cores", "value": 4 }, "current_usage": { "cpu": { "mean": 1.408104237368086, "std": 3.646794421903069, "unit": "%s" }, "disk": { "unit": "GB", </pre>

	<pre>===== EJECUTANDO ACCION DE PRUEBA (placement) ===== --- Enviando solicitud a VM Placement --- { "cpu": 64, "ram_gb": 1.0, "disk_gb": 1.0, "zone": "AZ4", "platform": "openstack", "user_profile": "Estudiante", "technical_context": "Virtualizacion general" } --- Respuesta de VM Placement --- { "error": "No hay hosts disponibles que cumplan los requisitos de riesgo", "success": false } ===== GET /nodes/status (despues) ===== { "server1": { "cpu_capacity": { "unit": "cores", "value": 4 }, "current_usage": { "cpu": { "mean": 1.8152565379510648, "std": 4.171483459885609, "unit": "%" }, "disk": { "unit": "GB", "used": 7.1510712320000005 }, "ram": { "mean": 1.9457124406012936, "std": 0.48910130116113865, "unit": "GiB" } }, "disk_capacity": { "unit": "GB", "value": 9.6 }, "enabled": true, "id": "server1", "in_maintenance": false, "metadata": { "datacenter": "Lima", "rack": "RACK-AZ1" }, "name": "compute-node-server1", }, } ===== COMPARANDO METRICAS ANTES VS DESPUES ===== No hubo cambios detectables en las metricas. ===== Evidencias generadas en carpeta: evidencias/nodes_status_2025-12-01_08-57-36 ===== ubuntu@app:~/vm_placement\$ █</pre>
Comentarios y/o consideraciones	-

6. Módulo 6: Persistence & Logging

6.1 Persistencia del estado del Slice en la Base de Datos

Número de caso	6.1
Objetivo del test	Validar que la base de datos registra correctamente el estado del slice durante su ciclo de vida, incluyendo creación, activación y eliminación, sin generar duplicados ni dejar registros huérfanos.
Diagrama de red del test	UI / API → Slice Manager → Módulo de Persistencia (BD)
Prerrequisitos	● Slice Manager operativo. ● Base de datos accesible y limpia (sin registros previos) ● Herramienta para consultar BD. ● Template válido para desplegar el slice.
Procedimiento	1. Consultar la tabla de slices antes del despliegue y registrar el número de entradas. 2. Crear un slice desde la UI.

	3. Consultar de nuevo la BD: 4. Eliminar el slice 5. Consultar de nuevo la BD:
Resultados esperados	● El registro del slice aparece al crearse. ● Su estado cambia correctamente a <i>active</i>. ● Al eliminarlo, cambia a <i>deleted</i> o desaparece. ● No se generan duplicados ni estados inválidos.
Resultado del test	PASS: El estado del slice fue registrado y actualizado correctamente en la base de datos durante todo su ciclo de vida. FAIL: Persisten datos inconsistentes (duplicados, estados incorrectos) o el registro no aparece o no cambia estado correctamente. NT: No se pudo ejecutar la prueba por indisponibilidad del Slice Manager o de la BD.
Evidencias	● Capturas del contenido de la tabla de slices antes/después.  The evidence includes three terminal screenshots showing the command 'ps aux grep qemu' being executed on three different servers (server1, server2, server3) and two screenshots of the phpMyAdmin database interface showing the 'slices' table. The terminal outputs show the following results: <pre> ubuntu@server1:~\$ ps aux grep qemu ubuntu 226994 0.0 0.0 8160 720 pts/2 S+ 05:19 0:00 grep --color=auto qemu ubuntu@server2:~\$ ps aux grep qemu ubuntu 194369 0.0 0.0 8160 724 pts/1 S+ 05:19 0:00 grep --color=auto qemu ubuntu@server3:~\$ ps aux grep qemu ubuntu 193460 0.0 0.0 8160 656 pts/3 S+ 05:19 0:00 grep --color=auto qemu </pre> The phpMyAdmin screenshots show the 'slices' table with columns: id, name, state, created_at, updated_at, deleted_at, description, created_by, updated_by. The table contains 25 rows of data, all with state 'active' and deleted_at set to null.

[illegible]

Despues:

The screenshot shows the 'Run' configuration dialog in PyCharm. The 'Environment' tab is active, showing a list of system variables. The 'Python interpreter' is set to 'Python 3.7.4 Shell (64-bit)'. The 'Run' button is highlighted.

The screenshot displays the phpMyAdmin interface for a database named 'phpmyadmin'. The main content area shows the structure and data of the 'tbl_users' table. The table has the following columns: id, first_name, last_name, email, phone, password, and status. The data is as follows:

id	first_name	last_name	email	phone	password	status
1	John	Doe	john.doe@example.com	1234567890	1234567890	1
2	Jane	Smith	jane.smith@example.com	9876543210	9876543210	1
3	Bob	Johnson	bob.johnson@example.com	5678901234	5678901234	1
4	Alice	Williams	alice.williams@example.com	4321098765	4321098765	1
5	Charlie	Brown	charlie.brown@example.com	3210987654	3210987654	1
6	Diana	Green	diana.green@example.com	2109876543	2109876543	1
7	Frank	Miller	frank.miller@example.com	1098765432	1098765432	1
8	Grace	Wilson	grace.wilson@example.com	0987654321	0987654321	1
9	Henry	Moore	henry.moore@example.com	9876543210	9876543210	1
10	Ivy	Taylor	ivy.taylor@example.com	8765432109	8765432109	1
11	Jack	Anderson	jack.anderson@example.com	7654321098	7654321098	1
12	Karen	Thomas	karen.thomas@example.com	6543210987	6543210987	1
13	Liam	Clark	liam.clark@example.com	5432109876	5432109876	1
14	Mia	Lewis	mia.lewis@example.com	4321098765	4321098765	1
15	Noah	Walker	noah.walker@example.com	3210987654	3210987654	1
16	Olivia	Hall	olivia.hall@example.com	2109876543	2109876543	1
17	Peter	Young	peter.young@example.com	1098765432	1098765432	1
18	Quinn	King	quinn.king@example.com	0987654321	0987654321	1
19	Rachel	Wright	rachel.wright@example.com	9876543210	9876543210	1
20	Samuel	Scott	samuel.scott@example.com	8765432109	8765432109	1

The screenshot displays the AWS IAM console interface. At the top, the 'Groups' page is selected. Below the navigation pane, the 'Groups' list table is shown. The table has columns for Name, Type, Path, Permissions, and Attached Policies. The 'AWS-ReadOnlyAccess' group is highlighted, showing it is a managed group with the path 'AWS-managed' and is attached to the 'AWSReadOnlyAccessPolicy'.

- Logs de creación y eliminación del slice.

Comentarios y/o consideraciones

Se tiene en consideración en tiempo real del registro del Log

6.2 Registro de Logs al Crear/Eliminar slice

Número de caso	6.2
Objetivo del test	Verificar que el sistema genera correctamente entradas de log durante la creación y eliminación del slice, incluyendo información del proceso, errores y finalización.
Diagrama de red del test	Slice Manager → Módulo de Logging → Archivo de logs
Prerrequisitos	● Slice Manager habilitado para escribir logs. ● Archivo de logs vacío o rotado. ● Acceso al directorio donde se almacenan los logs.
Procedimiento	1. Rotar o limpiar el archivo de logs antes del test. 2. Crear un slice mediante UI. 3. Verificar que se escribe una entrada de log. 4. Eliminar el slice. 5. Verificar que se registra una entrada de eliminación. 6. Enviar un JSON inválido y verificar que se registre un error.
Resultados esperados	● El archivo de logs contiene entradas de creación y eliminación del slice. ● Los errores y mensajes del flujo quedan registrados. ● Cada evento incluye timestamp y nivel de log.
Resultado del test	PASS: Los logs registraron correctamente la creación y eliminación del slice con la información esperada. FAIL: No aparecen entradas de log, faltan eventos, o los mensajes no corresponden al proceso ejecutado. NT: No se ejecutó la prueba por falta de acceso al archivo de logs o el Slice Manager estaba inactivo.
Evidencias	● Capturas del archivo de log antes/después. ● Ejemplos de mensajes INFO y ERROR.
Comentarios y/o consideraciones	Se ve que el registro queda persistente en la base de datos.

7. Módulo 7: Monitoreo

7.1 Disponibilidad de Node Exporter en los nodos del clúster

Número de caso	7.1
----------------	-----

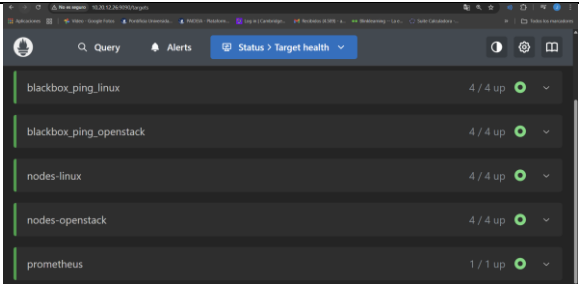
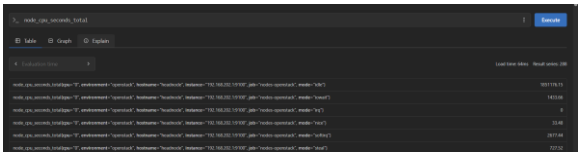
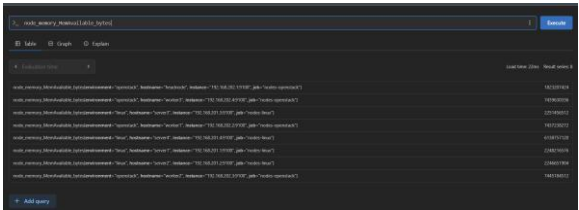
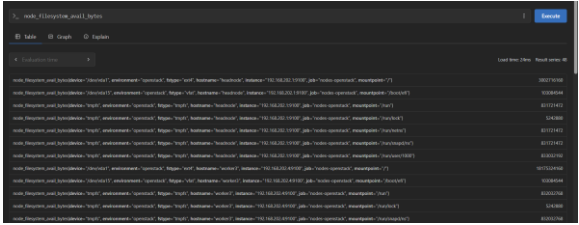
Objetivo del test	Validar que cada nodo del clúster (Linux y/o OpenStack) expone correctamente las métricas básicas de sistema mediante Node Exporter en el puerto 9100, garantizando que el servicio está operativo y accesible para Prometheus.
Diagrama de red del test	Worker/Server → Node Exporter → HTTP:9100/metrics
Prerrequisitos	• Node Exporter instalado y configurado en cada nodo a monitorear. • Servicio de Node Exporter activo (systemctl status node_exporter). • Conectividad hacia el nodo (puerto 9100 abierto). • IP de cada nodo identificada mediante:
Procedimiento	1. Verificar el estado del servicio Node Exporter en el nodo: sudo systemctl status node_exporter 2. Desde el servidor de monitoreo (o cualquier host con acceso), ejecutar: curl http://<IP_NODO>:9100/metrics Ej: http://192.168.202.1:9100/metrics 3. Comprobar que la respuesta contiene métricas, por ejemplo: - node_cpu_seconds_total - node_memory_MemAvailable_bytes - node_filesystem_avail_bytes 4. Repetir los pasos 1–3 para todos los nodos definidos en el clúster (Linux y OpenStack si aplican).
Resultados esperados	• El servicio node_exporter aparece como active (running) en todos los nodos. • La URL http://<IP_NODO>:9100/metrics responde sin errores HTTP. • Se observan métricas del tipo node_* en la salida, indicando que el nodo está reportando CPU, memoria y disco.
Resultado del test	PASS: Todos los nodos responden en :9100/metrics y el servicio Node Exporter está activo. FAIL: Algún nodo no tiene el servicio activo, no responde en el puerto 9100 o no devuelve métricas válidas.

	NT: No se ejecutó la prueba por indisponibilidad de los nodos o problemas de red previos.
Evidencias	• Capturas de systemctl status node_exporter en al menos un worker y/o nodo OpenStack. <pre> ubuntu@server1:~\$ systemctl status node_exporter ● node_exporter.service - Prometheus Node Exporter Loaded: loaded (/etc/systemd/system/node_exporter.service; enabled; vendor preset: enabled) Active: active (running) since Tue 2025-11-11 01:09:17 UTC; 2 weeks 6 days ago Main PID: 20259 (node_exporter) Tasks: 8 (limit: 4602) Memory: 21.9M CGroup: /system.slice/node_exporter.service └─20259 /usr/local/bin/node_exporter ubuntu@server1:~\$ </pre> • Capturas de la salida de: curl http://<IP_NODO>:9100/metrics donde se ven algunas métricas node_*. <pre> ubuntu@server1:~\$ curl http://192.168.201.1:9100/metrics # HELP go_gc_duration_seconds A summary of the pause duration of garbage collection cycles. # TYPE go_gc_duration_seconds summary go_gc_duration_seconds{quantile="0"} 2.687e-05 go_gc_duration_seconds{quantile="0.25"} 4.849e-05 go_gc_duration_seconds{quantile="0.5"} 6.987e-05 go_gc_duration_seconds{quantile="0.75"} 8.358e-05 go_gc_duration_seconds{quantile="1"} 0.00119357 go_gc_duration_seconds_sum 10.385119461 go_gc_duration_seconds_count 37312 # HELP go_goroutines Number of goroutines that currently exist. # TYPE go_goroutines gauge go_goroutines 0 # HELP go_info Information about the Go environment. # TYPE go_info gauge go_info{version="go1.21.4"} 1 # HELP go_memstats_alloc_bytes Number of bytes allocated and still in use. # TYPE go_memstats_alloc_bytes gauge go_memstats_alloc_bytes 2.766104e+06 # HELP go_memstats_alloc_bytes_total Total number of bytes allocated, even if freed. # TYPE go_memstats_alloc_bytes_total counter go_memstats_alloc_bytes_total 5.9969154984e+10 # HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table. # TYPE go_memstats_buck_hash_sys_bytes gauge go_memstats_buck_hash_sys_bytes 2.023359e+06 # HELP go_memstats_frees_total Total number of frees. # TYPE go_memstats_frees_total counter go_memstats_frees_total 8.14445683e+08 # HELP go_memstats_gc_sys_bytes Number of bytes used for garbage collection system metadata. # TYPE go_memstats_gc_sys_bytes gauge go_memstats_gc_sys_bytes 4.39486e+06 # HELP go_memstats_heap_alloc_bytes Number of heap bytes allocated and still in use. </pre>
Comentarios y/o consideraciones	• Node Exporter reporta más de 700 métricas; basta con validar la presencia de las esenciales (CPU, RAM, disco). • Si un nodo no está orientado a cómputo (ej. headnode), igual debe monitorearse para obtener carga general del clúster.

7.2 Validación de la recolección de métricas por Prometheus

Número de caso	7.2
Objetivo del test	Verificar que Prometheus está recolectando correctamente las métricas expuestas por cada nodo del clúster mediante Node Exporter, confirmando que las consultas (query y query_range) funcionan y que las series de tiempo se almacenan durante el periodo esperado.
Diagrama de red del test	Nodos (Node Exporter :9100) ▼ Prometheus (Scraper :9090) ▼ Consultas: /api/v1/query /api/v1/query_range

Prerrequisitos	● Cada nodo debe tener Node Exporter funcionando correctamente (validado en el test 7.1). ● Prometheus instalado y ejecutándose ● Archivo prometheus.yml contiene targets de todos los nodos ● Conectividad desde Prometheus a los puertos 9100
Procedimiento	1. Verificar estado del servicio Prometheus 2. Desde el navegador o curl, validar acceso local: curl http://IP:9090/-/healthy 3. Consultar una métrica en tiempo real para un nodo: curl "http://IP:9090/api/v1/query?query=node_cpu_seconds_total" 4. Confirmar que todos los targets aparecen en GREEN en: http://<IP>:9090/targets
Resultados esperados	● El servicio Prometheus está activo y sano (healthy). ● Todas las instancias de Node Exporter aparecen en estado UP en la interfaz /targets. ● Las consultas HTTP devuelven datos, no errores. ● Las queries con query_range muestran series temporales válidas. ● Se observan métricas válidas con prefijo node_* en CPU, RAM y Disco.
Resultado del test	PASS: Todos los targets están UP y las consultas devuelven valores consistentes. FAIL: Algún target aparece DOWN o las queries devuelven series vacías o errores HTTP. NT: Prueba no ejecutada por indisponibilidad del servicio o nodos.
Evidencias	● Captura de status prometheus.  ● Screenshot de /targets mostrando todos UP.

	 ● Capturas de gráficas o salida JSON de queries: - node_cpu_seconds_total  - node_memory_MemAvailable_bytes  - node_filesystem_avail_bytes 
Comentarios y/o consideraciones	● Si un nodo no aparece en Prometheus, validar: - sintaxis YAML incorrecta - error de indentación - firewall o puertos bloqueados - caída del Node Exporter ● Prometheus almacena datos en /prometheus/ → tamaño depende de la retención/tiempo configurada.

7.3 Validación de la API de Monitoreo

Número de caso	7.3
Objetivo del test	Validar que el servicio metrics_api.py expone correctamente las métricas históricas por nodo

	y el JSON consolidado nodes_status.json, permitiendo que otros módulos (como Placement) consuman datos de CPU, RAM y disco sin interactuar directamente con Prometheus.
Diagrama de red del test	Prometheus ←scraping— Node Exporter metrics_api.py (HTTP:5001) nodes_status.json y endpoints REST
Prerrequisitos	● Prometheus operativo y recolectando métricas. ● Script metrics_api.py ejecutándose en el puerto 5001. ● Archivo nodes_status.json generado mediante generate_nodes_status.py.
Procedimiento	1. Verificar que la API está activa curl http://localhost:5001/nodes/status 2. Consultar métricas históricas de un nodo. Ej: curl http://localhost:5001/metrics/server1 3. Verificar cálculo de estadísticas (media, std, unidad) cat nodes_status.json 4. Validar consistencia entre API y JSON
Resultados esperados	● /nodes/status responde correctamente. ● /metrics/<node> responde con data histórica válida. ● Las unidades están estandarizadas (GiB, GB, %). ● No hay errores 404 ni 500 en la API.
Resultado del test	PASS: Ambos endpoints responden sin error y los datos son coherentes. FAIL: Algún nodo no aparece, falla la API o el JSON está corrupto. NT: No se realizó la prueba.
Evidencias	● Captura de curl http://localhost:5001/nodes/status

	<pre> ubuntu@app:~/monitoring/metrics_api\$ curl http://localhost:5001/nodes/status jq . { "Total": 180, "Received": 3054, "kferd": 0, "Average Speed": 0, "Time": 0, "Time Current": 0, "Dload": 0, "Upload": 0, "Total": 0, "Spent": 0, "Left": 0, "Speed": 0, "server1": { "cpu_capacity": { "unit": "cores", "value": 4 }, "current_usage": { "cpu": { "mean": 1.7541069842166413, "std": 3.2030296417078046, "unit": "%", "value": 0 }, "disk": { "unit": "GB", "value": 7.151067136000001 }, "ram": { "mean": 2.6301816697700375, "std": 0.5540743398292477, "unit": "GiB", "value": 0 } }, "disk_capacity": { "unit": "GB", "value": 9.6 }, "enabled": true, "in_maintenance": false, "is_server1": false, "metadata": { "platform": "linux" } } } </pre> • Extracto de nodes_status.json con los valores de un nodo. <pre> ubuntu@app:~/monitoring/metrics_api\$ cat nodes_status.json { "server1": { "id": "server1", "name": "compute-node-server1", "platform": "linux", "zone": "AZ1", "cpu_capacity": { "value": 4, "unit": "cores" }, "ram_capacity": { "value": 3.8, "unit": "GiB" }, "disk_capacity": { "value": 9.6, "unit": "GB" }, "current_usage": { "cpu": { "mean": 1.7541069842166413, "std": 3.2030296417078046, "unit": "%", "value": 0 }, "ram": { "mean": 2.6301816697700375, "std": 0.5540743398292477, "unit": "GiB", "value": 0 }, "disk": { "used": 7.151067136000001, "unit": "GB" } }, "enabled": true, "in_maintenance": false, "metadata": { "platform": "linux" } } } </pre>
Comentarios y/o consideraciones	• Esta API permite desacoplar completamente la lógica de monitoreo del placement. • Se evita que el módulo de placement consulte Prometheus directamente. • Facilita exportar histórico en JSON para análisis offline o predicción futura.

7.3¿4 Integración del Monitoreo con el Módulo de VM Placement

Número de caso	7.4
Objetivo del test	Validar que el módulo de Placement obtiene correctamente las métricas históricas consolidadas desde el servicio de monitoreo (metrics_api) mediante el endpoint /nodes/status y utiliza estos valores (mean, std, disk used) para alimentar el algoritmo probabilístico de selección de host.

Diagrama de red del test	Node Exporter → Prometheus → metrics_api.py → nodes_status.json → Placement API
Prerrequisitos	● Prometheus está recolectando métricas de todos los nodos (server1–4, worker1–3). ● El servicio metrics_api.py está corriendo ● El archivo nodes_status.json fue generado. ● La API de Placement (api_placement_handler.py) está ejecutándose ● El algoritmo de Placement está configurado para consumir: GET /nodes/status
Procedimiento	1. Verificar que el JSON consolidado está disponible desde Monitoreo 2. Iniciar el módulo de Placement 3. Realizar una solicitud de Creation / VM Placement. 4. Verificar que el módulo de Placement obtiene métricas desde el JSON. 5. Confirmar que el algoritmo evalúa riesgo y selecciona un host.
Resultados esperados	● La API de Placement recupera automáticamente las métricas desde metrics_api. ● Los valores de CPU, RAM y disco usados coinciden con los presentes en nodes_status.json. ● El algoritmo ejecuta el modelo probabilístico (media, varianza) sin error. ● Se obtiene un host válido en la zona solicitada.
Resultado del test	PASS: La solicitud de placement usa métricas en tiempo casi real y selecciona un host. FAIL: El placement no puede leer /nodes/status, retorna errores 500 o los datos llegan incompletos. NT: No se ejecutó la prueba por indisponibilidad de métricas o Prometheus
Evidencias	● Log de Placement mostrando valores media y varianza:

	<pre>ubuntu@app:~/monitoring/metrics_api\$ curl http://localhost:5001/nodes/status jq '.worker1' % Total % Received % Xferd Average Speed Time Time Time Current Dload Upload Total Spent Left Speed 100 3062 100 3062 0 0 747k 0 0 --:--:-- --:--:-- --:--:-- 747k { "cpu_capacity": { "unit": "cores", "value": 4 }, "current_usage": { "cpu": { "mean": 3.6128282185833285, "std": 5.192863224116261, "unit": "ms" }, "disk": { "unit": "GB", "used": 8.898689152 }, "ram": { "mean": 1.3483686852848329, "std": 8.7188309983668148, "unit": "GiB" } }, "disk_capacity": { "unit": "GB", "value": 25 }, "enabled": true, "id": "worker1", "in_maintenance": false, "metadata": { "datacenter": "lima", "rack": "BACK-AZ4" }, "name": "compute-node-worker1", "platform": "openstack", }</pre> - Evidencia del placement exitoso <pre>ubuntu@app:~/placement\$./vm_placement.py ubuntu@app:~/placement\$ python3 api_placement_handler.py SERVIDOR DE VM PLACEMENT API Indicadores disponibles: POST /api/v1/placement - Solicitar placement de VM GET /api/v1/health - Health check # Iniciando servidor en http://localhost:5000 \$ Serving FastAPI app 'api_placement_handler' \$ Using Model 'api_placement_handler' WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead. \$ Running on all addresses (0.0.0.0) \$ Running on http://0.0.0.0:5000 \$ Running on https://0.0.0.0:5001 Press CTRL+C to quit \$ Restarting with stat SERVIDOR DE VM PLACEMENT API Indicadores disponibles: POST /api/v1/placement - Solicitar placement de VM GET /api/v1/health - Health check # Iniciando servidor en http://localhost:5000 \$ Debugger is active! \$ Debugger PID: 1107800 \$ Detected change in /home/ubuntu/vm_placement/test_data/platforms_auto.py, reloading. \$ Restarting with stat SERVIDOR DE VM PLACEMENT API Indicadores disponibles: POST /api/v1/placement - Solicitar placement de VM GET /api/v1/health - Health check</pre> - Evidencia de la respuesta JSON al usuario
Comentarios y/o consideraciones	- Esta API permite desacoplar completamente la lógica de monitoreo del placement. - Se evita que el módulo de placement consulte Prometheus directamente. - Facilita exportar histórico en JSON para análisis offline o predicción futura.